@techwithvishalraj

Java

Multithreding

Follow

Share

@techwithvishalraj

Introduction

- Multithreading in Java is a process of executing multiple threads simultaneously.
- A thread is a lightweight sub-process, the smallest unit of processing.
- Multiprocessing and multithreading, both are used to achieve multitasking.
- However, we use multithreading than multiprocessing because threads use a shared memory area. They don't allocate separate memory area so saves memory, and context-switching between the threads takes less time than process.

Advantages of Java Multithreading

- It doesn't block the user because threads are independent and you can perform multiple operations at the same time.
- You can perform many operations together, so it saves time.
- Threads are independent, so it doesn't affect other threads if an exception occurs in a single thread.

Multitasking

- Multitasking is a process of executing multiple tasks simultaneously. We use multitasking to utilize the CPU. Multitasking can be achieved in two ways:

  1) Process-based Multitasking (Multiprocessing)
  2) Thread-based Multitasking (Multithreading)

@techwithvishalraj

Process-based Multitasking (Multiprocessing)

- Each process has an address in memory. In other words, each process allocates a separate memory area.
- A process is heavyweight.
- Cost of communication between the process is high.
- Switching from one process to another requires some time for saving and loading registers, memory maps, updating lists, etc.

Thread-based Multitasking (Multithreading)

- Threads share the same address space.
- A thread is lightweight.
- Cost of communication between the thread is low.
- At least one process is required for each thread.

@techwithvishalraj

Program vs Process vs Thread

Program:

- **A program is an executable file containing a set of instructions passively stored on disk.**

Process:

- **A process is a program in execution.**
- **When a program is loaded into the memory and becomes active, the program becomes a process.**
- **A process requires some essential resources such including CPU time, program counter, stack, memory, files, and I/O devices — to accomplish its task.**
- **Program is a passive entity while process is an active entity.**
- **One program can have multiple processes.**

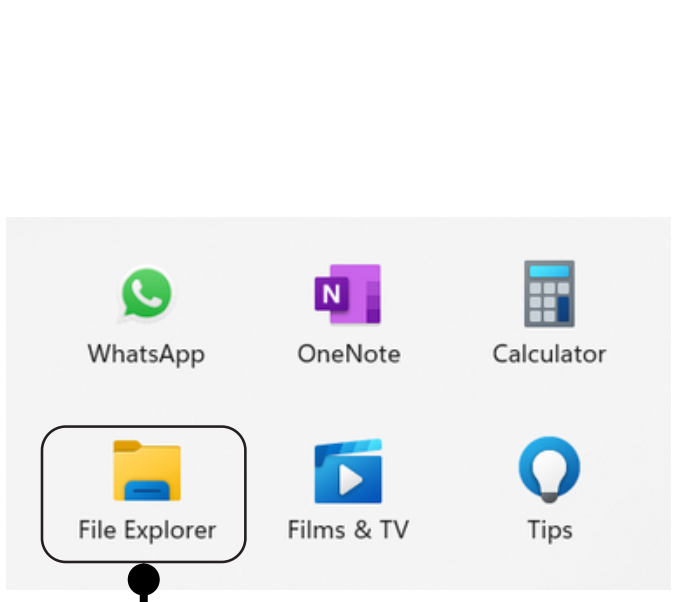
Thread :

- **A Thread is the smallest unit of execution within a process (or) basically it is a segment of a process .**
- **Thread is also known as lightweight process.**
- **There are two types of processes :**
  1. **Single Threaded Process**
  2. **Multi Threaded Process**

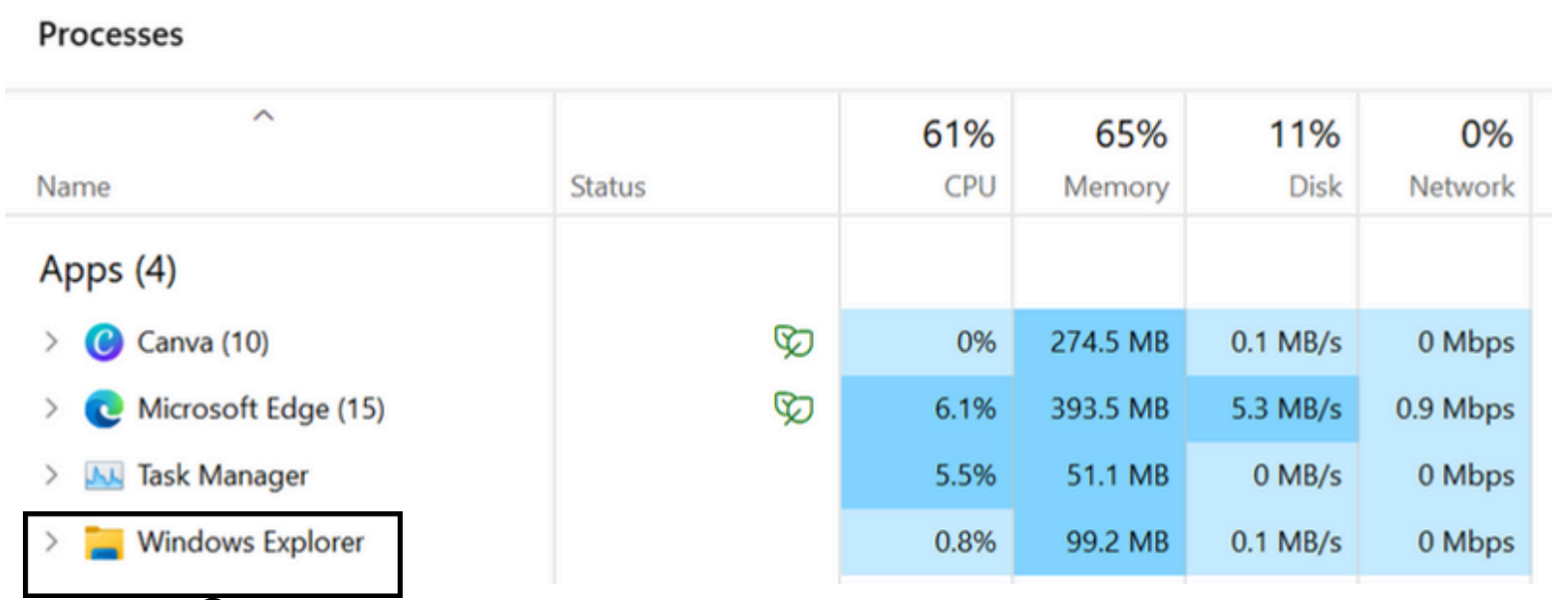
@techwithvishalraj

Program vs Process vs Thread

As shown in the figure, a thread is executed inside the process. There is context-switching between the threads. There can be multiple processes inside the OS, and one process can have multiple threads.

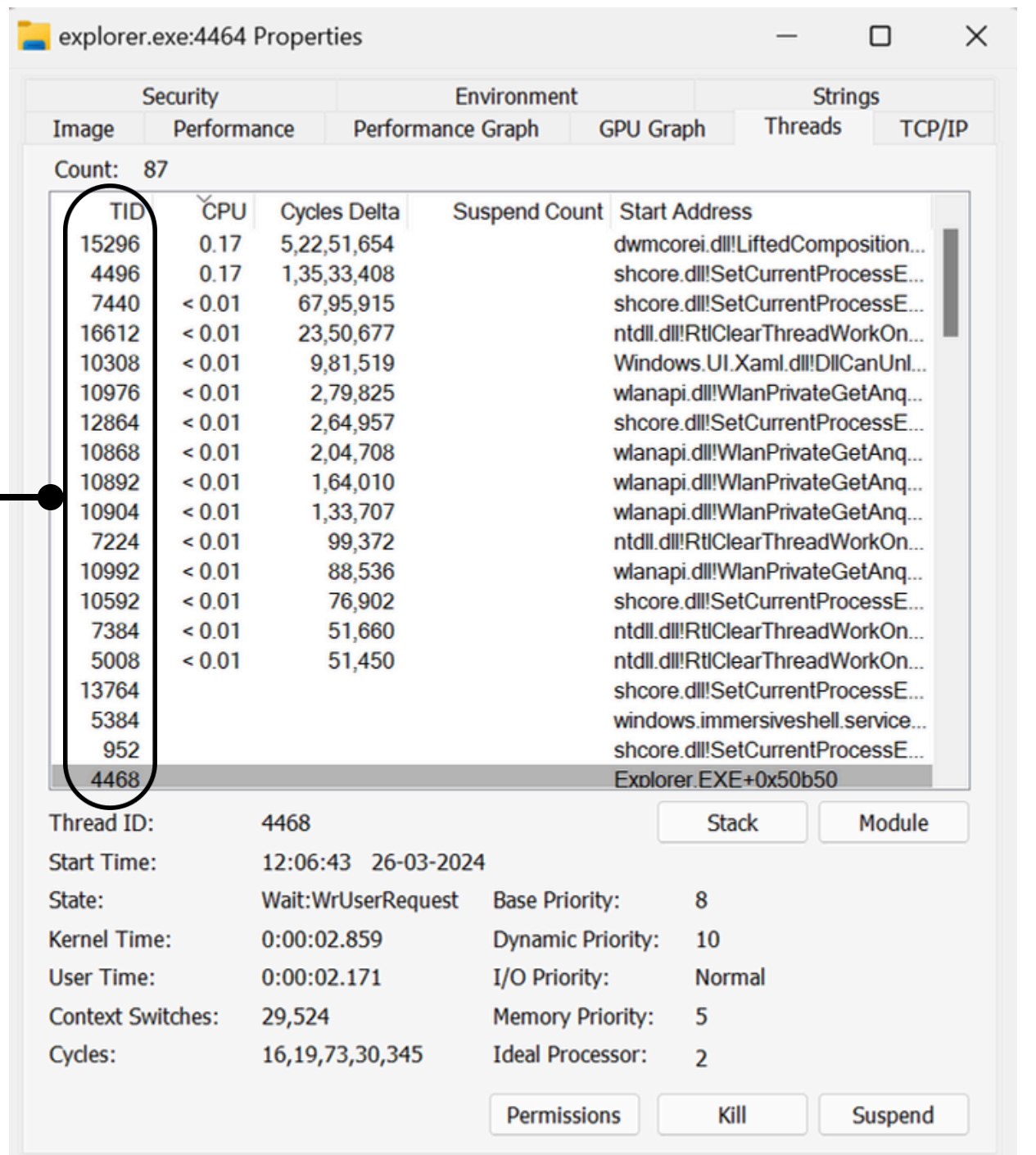


Program (Disk)



process (RAM)

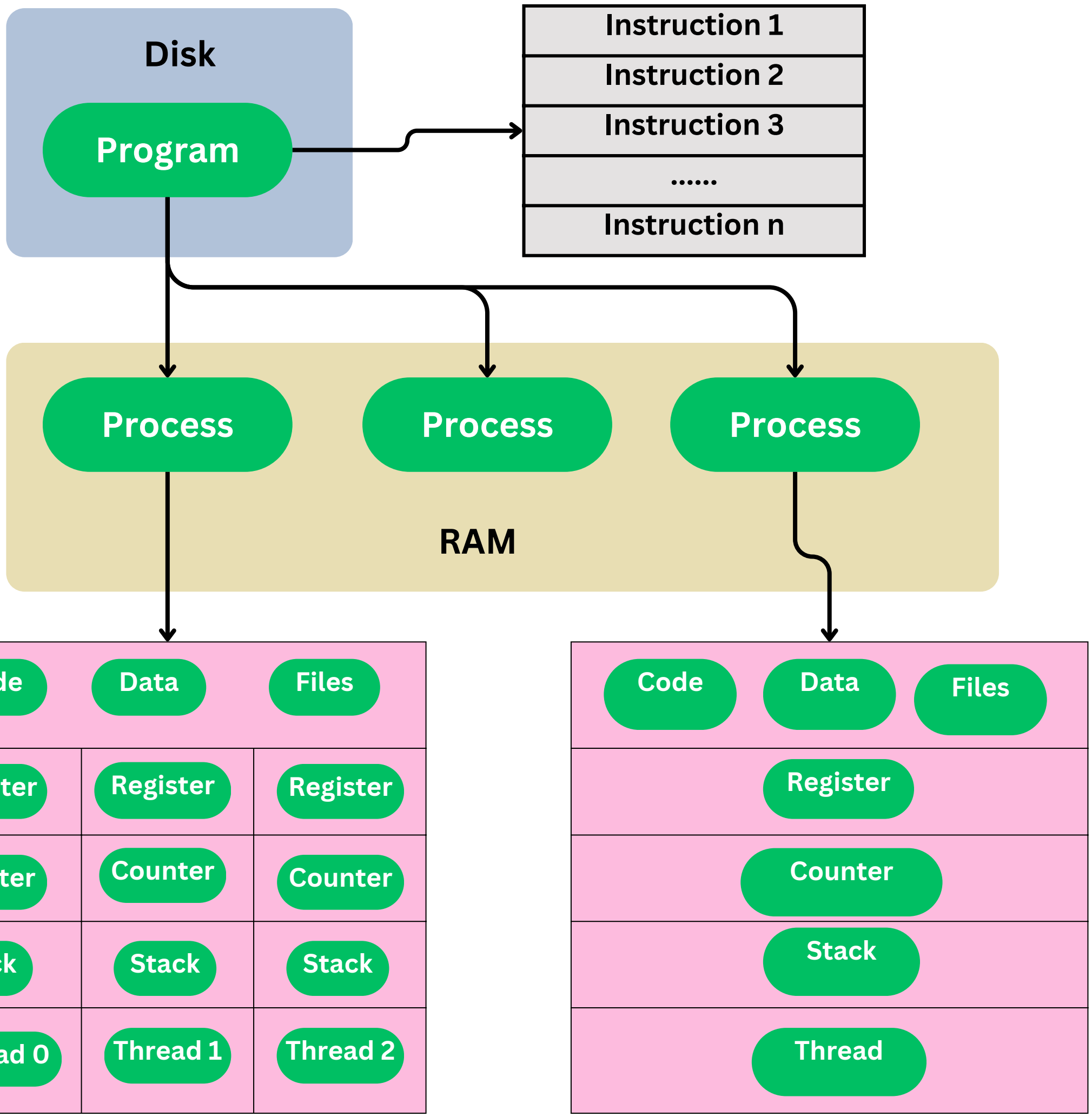
Threads
TID - Thread id



Note: Install process explorer to see the threads and process.
https://process-explorer

@techwithvishalraj

Program vs Process vs Thread



Multi Threaded Process

Single Threaded Process

Single Threaded Process

- A single thread executes the instructions line by line from beginning to end.
- A single-threaded process has one program counter specifying the next instruction to execute.
- The execution of such a process must be sequential — the CPU executes one instruction after another, until the process completes.

Multi Threaded Process

- Multithreading is a model of program execution that allows for multiple threads to be created within a process, executing independently but concurrently sharing the process resources like data, memory, resources, files, etc with their peer threads within a process.
- Each thread has it’s own its own stack, register and program counters .
- Threads can directly communicate with each other as they share the same address space.

@techwithvishalraj

How to create a Thread

- A Thread is the smallest unit of execution within a process (or) basically it is a segment of a process .
- Thread is also known as lightweight process.
- There are two ways to create a thread:
  1. By extending the Thread class
  2. By implementing Runnable interface.

@techwithvishalraj

1. By Extending the Thread Class

Thread class:

- **Thread class provide constructors and methods to create and perform operations on a thread. Thread class extends Object class and implements Runnable interface.**
- **Commonly used Constructors of Thread class:**
  1. **Thread()**
  2. **Thread(String name)**
  3. **Thread(Runnable r)**
  4. **Thread(Runnable r, String name)**

```
class CustomThread extends Thread {
   public void run() {
      for (int i = 0; i < 10; i++) {
         System.out.println("i= " + i);
      }
   }
}
public class ByExtendingThreadClass {
   public static void main(String[] args) {
      CustomThread t1 = new CustomThread();
      t1.setName("CustomThread");
      t1.start();
      CustomThread t2 = new CustomThread();
      t2.start();
   }
}
```

```
Output:
i= 0
i= 1
i= 0
i= 1
i= 2
i= 3
i= 2
i= 4
i= 5
i= 6
i= 7
i= 3
i= 4
i= 8
i= 9
i= 5
i= 6
i= 7
i= 8
i= 9
```

2. By Implementing the Runnable Interface

@techwithvishalraj

- **Another way to create a thread is by implementing the Runnable interface. This is preferred when your class is already extending another class since Java does not support multiple inheritance.**

- **If you are not extending the Thread class, your class object would not be treated as a thread object. So you need to explicitly create the Thread class object. We are passing the object of your class that implements Runnable so that your class run() method may execute.**

```
class CustomRunnable implements Runnable{
   public void run() {
      for (int i = 0; i < 10; i++) {
         System.out.println("i= " + i);
      }
   }
}
public class ByImplementingRunnableInterface {
   public static void main(String[] args) {
      Thread t1 =new Thread(new CustomRunnable());
      t1.start();
      Thread t2 =new Thread(new CustomRunnable());
      t2.start();
   }
}
```

```
Output:
i= 0
i= 1
i= 0
i= 1
i= 2
i= 3
i= 2
i= 4
i= 5
i= 6
i= 7
i= 3
i= 4
i= 8
i= 9
i= 5
i= 6
i= 7
i= 8
i= 9
```

@techwithvishalraj

Thank you!

vishal-bramhankar

techwithvishalraj

Vishall0317